
Name

windetect — Tk window detection library

Synopsis

```
package require windetect
::windetect::configure ?option? ?option value? ?option value?
::windetect::detect ?state?
::windetect::exit
::windetect::instrument ?bindtag? ?eventlist?
```

Description

This reference page corresponds to windetect version 2.0

The package windetect is a library that provides the detection functionality for tracking windows in a Tk application using a pointer device (for example a mouse, which will be presumed from this point in the text onward). Each time when the mouse pointer finds itself inside a new window, either by the mouse pointer being moved across the screen or by changes to the GUI, windetect reports the following data: the pathname of the new pointer window and the screen coordinates of the mouse pointer. The reporting mechanism is a callback command, to which these data are passed as parameters.

Windetect is meant to be loaded into the interpreter that executes the code for the GUI whose windows need to be detected (the *client interpreter*). Immediately after being loaded, the detection functionality isn't active yet. The following two steps need to be taken to enable the detection functionality:

1. configuring the sensitivity of the windows in the GUI, using the command `::windetect::instrument`. This step is also known as *instrumentation*.
2. activating the detection functionality by invoking `::windetect::detect on`

The commands in the package windetect are encapsulated in the namespace `::windetect`. This namespace does not export its commands; fully qualified command names have to be used.

Requirements

Tk version

Windetect requires Tk 8.5 or newer.

Windowing system / Operating system

Windetect functions overall well with the X11 windowing system (tested on Linux with various window managers and desktop environments), and with the windowing system supplied by the Microsoft Windows operating system. Proper functioning has not been tested with other windowing systems or operating systems, and therefore no claims are made about it.

Command syntax

```
::windetect::configure ?option? ?option value? ?option value?
```

Queries or modifies settings for the behaviour of windetect. If no arguments are supplied, the current settings for all configurable options are returned. If a single

option name is supplied, then the value for that particular option is returned. If one or more option-value pairs are supplied, the options are set to the newly supplied values. Modifying settings can only be done while the detection functionality is turned off.

The following options are supported:

-callback *cmdprefix*

Defines the callback command that is called when a new window is detected. The actual command called is the command prefix *cmdprefix* with the following three extra arguments appended:

- the pathname of the newly detected window
- the x coordinate of the mouse pointer relative to the root window of the screen
- the y coordinate of the mouse pointer relative to the root window of the screen

Therefore, the programmer needs to ensure that the callback command accepts at least three arguments additionally to what has been specified with *cmdprefix*.

By default, windetect uses a callback command that writes the values for these three variables to stdout.

-delay *ms*

Sets the time interval between the first detection event (after a previous reporting), and the subsequent reporting; also called the *reporting delay*. Such a delay causes detection events that occur in quick succession to be skipped from reporting. This spares the user from a possible overload of information. On the other hand, if the delay is chosen too long, a reporting may miss relevant detection events.

The value for *ms* must be a non-negative integer. If set to zero, reporting occurs for every single detection event. See [the paragraph “Toplevel crossings with zero or near-zero reporting delay”](#) below for limitations when using values for *ms* that are zero or near zero. For the specific purpose of skipping detection events that occur faster than what the human eye can follow, a reporting delay of a few tens of milliseconds is recommended. The default value is 50.

-wsrespite *ms*

This option is intended to overcome the issues, caused by latency of the windowing system, as described in [the paragraph “Toplevel crossings with zero or near-zero reporting delay”](#). The value for *ms* must be a non-negative integer. A useful value for *ms* should be based on the latency that the windowing system exhibits.

Windetect provides an auxiliary tool `wsrespite.guess` that inspects these latencies, and which recommends a suitable value for the option `-wsrespite`.

This program is to be found in the `utils` sub-directory of the windetect distribution. The default value for *ms* is zero.

::windetect::detect *?state?*

Activates or deactivates the detection functionality, depending on the value of *state*. Valid values for *state* are `on` and `off`. If no argument is supplied, the current detection state is returned. The initial state, immediately after loading the package, is `off`. If nothing has been instrumented, the command `detect on` returns an error.

::windetect::exit

Deactivates detection functionality, removes the `::windetect` namespace, and removes the package from the interpreter.

::windetect::instrument ?bindtag? ?event list?

This command configures the sensitivity of windows in the client GUI for detection by windetect. It does so by associating the events that windetect is based upon (Enter, Leave and Destroy) with binding tags.

If the command is invoked without arguments, it returns a dictionary containing the instrumentation information for the entire application. The keys of the dictionary are the binding tags currently used for instrumentation, and the values are lists of events associated with each binding tag.

If a single argument is specified, the command regards that as the *bindtag* argument. In that case, the command returns a list of events associated with *bindtag*. If *bindtag* is not used in the current GUI instrumentation, an empty list is returned.

If *bindtag* and *event list* are both specified, the command associates the events in *event list* with the binding tag *bindtag*, replacing any previous association for *bindtag*. An empty list for the argument *event list* removes the current association of events from the binding tag *bindtag*. The asterisk character *** can be used as a shorthand for the list of all three window events Enter, Leave and Destroy.

Instrumentation of a GUI

To participate in the detection process, windows in a GUI must be explicitly *instrumented*. This means that binding tags, carried by the windows in the GUI, are associated with the event handlers used internally by windetect. This is accomplished through a separate configuration command: `::windetect::instrument`.

A basic scenario, and probably the most common scenario in practice, is the general instrumentation of all windows in the client GUI (for all three window events that windetect is based upon). For this basic scenario, it suffices to issue `::windetect::instrument all *`. This command operates on the binding tag `all`, which Tk assigns to each widget upon creation.

Sometimes, a more specific instrumentation is needed. A GUI can be instrumented to any degree of specificity by carefully combining binding tags and detection events, and passing these combinations to `::windetect::instrument`. A few hints and examples of a more specific instrumentation follow:

- Suppose you wanted to instrument only one particular toplevel window `.tools` and all its descendants, then instead of targeting the binding tag `all`, you would issue `::windetect::instrument .tools *`.
- If you wanted to instrument all windows except one particular window `.tools.hammer`, then you could simply remove the standard binding tag `all` from the window `.tools.hammer`, along with the general instrumentation achieved by `::windetect::instrument all *`.
- Instrumenting a specific class of widgets can be done by letting `::windetect::instrument` operate on the binding tag for the widget class, which Tk assigns to each widget upon creation. For example: `::windetect::instrument Entry *`.
- The command `::windetect::instrument` doesn't add binding tags to the client GUI, nor does it remove any binding tags from it or change the original bindings. It only adds windetect's own bindings to the already existing bindings, or removes these bindings.

- The parameter *event list* is intended only for very specific situations where the detection mechanism needs to be insensitive to certain events on certain widgets.

After having completed the instrumentation, invoking `::windetect::detect` on will activate the detection functionality as configured.

Limitations and known issues

There are several limitations or issues, all of which occur under quite specific circumstances. The following paragraphs detail these issues and circumstances.

Grab activation and deactivation

The activation and the deactivation of a grab on a window does not result in reporting of a new pointer window if all of the following conditions are met:

- the Tk minor version is 8.6 or older, except patch levels 8.6.15 and later.
- the grab window is not a toplevel window
- at the time when the grab (de)activation takes place, the mouse pointer resides in a window outside the grab subtree.
- the grab activation or deactivation originates from calling the functions `Tk_Grab` or `Tk_Ungrab` in the Tk C-library. (If they originate from the invocation of the script level command `grab`, `windetect` works around the problem.)

This is a result of the issue addressed in ticket #47d4f29159 of the Tk source code management system.

Destruction of the pointer window

A new pointer window may remain undetected, or its pathname may be reported incorrectly upon destruction of the old pointer window. This issue is known to occur if the operating system is MS Windows (disregarding operating systems on which `windetect` has not been tested, see [the section “Requirements”](#)). Tk has been receiving fixes for this issue since patch level 8.6.12. See ticket #22349fc78a in the Tk source code management system for specifics.

Toplevel crossings with zero or near-zero reporting delay

Sometimes a Tk application unmaps, or changes the geometry or stacking order of a toplevel window, exposing another one at the position of the mouse pointer. A well-known example is the unmapping of a dialog after clicking the OK or Cancel button. In such situations, the X11 windowing system notifies Tk of relevant window events (including `Leave Enter` and `Destroy`), to which Tk responds in turn. These notifications may arrive late, and if the latency exceeds the reporting delay, `windetect` is bound to determine the new pointer window incorrectly. The chances of this happening increase with smaller values for the reporting delay, i.e. zero or near-zero.

The option `-wsrespite` is intended to remedy this issue if the reporting delay needs to be exactly zero. This option grants extra opportunity for Tk so that it may receive late windowing system notifications. (Note: without skipping detection events.) This technique often succeeds in preventing that `windetect` determines the new pointer window incorrectly. However, if the reporting delay does not need to be exactly zero, the issue can also be prevented by increasing the value for the reporting delay. Details regarding this issue, including the variability among windowing systems, are covered in [the section “Toplevel crossings and windowing system particulars”](#).

Testing

Windetect comes with two test scripts: `windetect-unit.test` and `windetect-gui.test`. The file `windetect-unit.test` exercises the windetect API without using the context of a GUI. By contrast, the tests in the file `windetect-gui.test` exercise the behaviour of windetect when tracking windows in a GUI, hence called *gui-integration tests*.

There are several aspects to be aware of when running the gui-integration tests:

- Quite some tests exercise a setup whereby toplevel windows overlap on the screen. These tests cannot be usefully run with tiling window managers. Unless the test option `-nowm` was provided to exclude these tests, the test script aborts if it detects a tiling window manager.
- When running the gui-integration tests under the circumstances as described in [the paragraph “Toplevel crossings with zero or near-zero reporting delay”](#), tests may fail because of latency of windowing system notifications. It's appreciated to report such test failures, but only if:
 - the test continues to fail with values for the option `-wsrespite` increased to well above what's recommended with the tool `wsrespite.guess`, and:
 - (for the x11 windowing system only) the failure exhibits with at least two different window managers.
- No windows from other applications should occupy the screen area where the test widgets are displayed.

Each test script is invoked as follows:

```
tcclsh test_script ?flag? ?option value? ...
```

This command line supports several optional arguments:

- a. configuration options that modify the behaviour of windetect. These are the same options as accepted by `::windetect::configure`, except for the option `-callback`. They are passed on to windetect.
- b. configuration options for the Tcl test harness. These are passed on to `tcclsh`.
- c. options and flags that modify the behaviour of the test script. These are only available to the test script `windetect-gui.test`, except for the option `-checkclean`, which is available to both test scripts.

-accommodatewm bool

Some window managers exhibit specific issues (other than the generic latency problems described in [the section “Toplevel crossings and windowing system particulars”](#)) that make certain gui-integration tests unreliable, or cause them to fail. For some of these issues, the test script `windetect-gui.test` knows how to work around them. This option specifies whether the test script should apply such accommodations. Beware that window manager accommodations are a very limited feature, based on best effort. The default value for `bool` is 1.

-checkclean

Presence of this flag on the command line makes the test script perform extra sanity checks for each test. If a check fails, the test script exits immediately with an error message. This option automatically enables the option `-detectiondetails`.

-detectiondetails

When testing the functionality of windetect proper, tests use an expected result that consists only of the newly detected window under the mouse pointer. This is the strict testing mode. Presence of the flag *-detectiondetails* induces a more detailed testing mode. In this mode, the expected result additionally includes the following details:

- the window event that induced the detection: Enter, Leave or Destroy
- the window that received the event

Besides using a different expected result, usage of this option also results in detection details being written to stdout, interspersed with the regular test output.

-extrawindowevents

Presence of this flag on the command line makes the test script output a notification for the following window events, additional to the usual Enter, Leave, and Destroy events: Configure, Map, Unmap, Visibility and Expose. This option is only valid in combination with the option *-detectiondetails*.

-eyefriendly

This flag makes the test script pause one second at appropriate times so that human vision can follow the changes to the screen.

-nowm

Presence of this flag on the command line makes the test script exclude those tests, where the window manager is involved in bringing about the detection events.

-skipintro

Presence of this flag on the command line makes the test script refrain from printing the introductory text consisting of the test environment and test configuration.

-timing

Presence of this flag on the command line makes the test script output timing information regarding the detection events and their reporting. The timing information is interspersed with the regular test output. Detection events are indicated with an asterisk (*) and reportings are indicated with a letter "r". Angle brackets ("<" and ">") enclose the events that belong to one and the same reporting.

-wmonly

Presence of this flag on the command line makes the test script run only those tests, where the window manager is involved in bringing about the detection events.

Toplevel crossings and windowing system particulars

Situations where a GUI changes the mapping state, geometry or stacking order of a toplevel window are special for windetect because they involve an extra process: the window manager. This extra process affects the timeliness of notifications sent by the windowing system to Tk, which is what the situations described in [the paragraph “Toplevel crossings with zero or near-zero reporting delay”](#) revolve around. Late

notifications only become an issue for windetect if it is configured for a reporting delay of zero or near zero. In those cases, they may make windetect determine the new pointer window incorrectly. This effect is best observed when running the test script `windetect-gui.test`, using zero as the value for the options `-delay` and `-wsrespite`, and passing the flag `-wmonly` to exclude tests that don't involve the window manager.

Considering the hardware of a computer system as a given, the latency of windowing system notifications varies considerably with the window manager in use. Beware that this variability not only exists between window managers of an entirely different make, but also between different releases, and even between different configurations of the same window manager, notably the setting for focus stealing prevention. On top of that, system load may add more erratic variability to the observed latencies.

The program `wsrespite.guess`, invoked with the parameter `-showstat` provides the user with information about the latencies observed with the gui-integration tests that involve toplevel transitions. It measures the time interval between an `Enter` event for the new pointer window and the immediately preceding `Leave`, or `Destroy` event for the previous pointer window, which is what specifically matters to windetect. (The maximum latency value observed in this inspection serves as a recommendation for the option `-wsrespite`.) This program resides in the `utils` sub-directory of the windetect distribution.

See also

`bindtags(n)`, `tcctest(n)`

Author

Erik Leunissen

Keywords

Detect and report windows under the mouse pointer.

License

Copyright © 2020 Erik Leunissen.

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at:

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.